

CO2 – AT3

Course Code: MLA0201

Name: Siva Balan K

Course Name: Fundamentals of Machine Learning

Reg.no: 192572037

Q1 — Linear Regression: High House Price Error

(i) Problem Identification

High error can result from outliers, poor feature selection, and unsuitable data.

(ii) Debugging

The dataset contains an extreme house-price value. RMSE is calculated to observe its effect.

(iii) Optimization

Remove the extreme outlier and retrain the regression model.

Code

```
import pandas as pd

from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

data = pd.DataFrame({
    "area": [600, 800, 1000, 1200, 1400, 1600, 1800, 2000, 2200, 2400],
    "rooms": [1, 2, 2, 3, 3, 4, 4, 5, 5, 6],
    "location_score": [5, 6, 6, 7, 7, 8, 8, 9, 9, 10],
    "price": [30, 40, 50, 60, 70, 80, 90, 100, 110, 500]
})

# Remove extreme outlier
data = data[data["price"] < 200]
```

```

X = data[["area", "rooms", "location_score"]]
y = data["price"]

model = LinearRegression()
model.fit(X, y)

pred = model.predict(X)
rmse = mean_squared_error(y, pred) ** 0.5

print("Q1: House Price Prediction")
print("Records after removing outlier:", len(data))
print("Optimized RMSE:", round(rmse, 2))

```

Output

```

----- RESTART: C:/Users/Siva Da
Q1: House Price Prediction
Records after removing outlier: 9
Optimized RMSE: 0.0
|

```

Interpretation

Removing the extreme outlier reduces prediction error and produces a more reliable regression model.

Q2 — Email Classification: Spam Misclassification

(i) Problem Identification

Spam can be classified as non-spam because of an unsuitable decision threshold or insufficient representation of spam words.

(ii) Debugging

The model's prediction probabilities are examined to identify uncertain classifications.

(iii) Optimization

Use TF-IDF features and a lower decision threshold to detect more spam.

Code

```
import pandas as pd

from sklearn.feature_extraction.text import TfidfVectorizer

from sklearn.linear_model import LogisticRegression
```

```
data = pd.DataFrame({
    "text": [
        "meeting scheduled tomorrow",
        "project report attached",
        "get free prize now",
        "win money click here",
        "team meeting at ten",
        "free offer claim prize",
        "class assignment submitted",
        "you won free cash"
    ],
    "label": [0, 0, 1, 1, 0, 1, 0, 1]
})
```

```
vectorizer = TfidfVectorizer()

X = vectorizer.fit_transform(data["text"])

y = data["label"]
```

```
model = LogisticRegression()

model.fit(X, y)
```

```
new_email = ["free prize click now"]
```

```
new_X = vectorizer.transform(new_email)
```

```
prob = model.predict_proba(new_X)[0][1]
```

```
prediction = 1 if prob >= 0.40 else 0
```

```
print("Spam Probability:", round(prob * 100, 2), "%")
```

```
print("Prediction:", "SPAM" if prediction else "NOT SPAM")
```

Output

```
Spam Probability: 62.89 %  
Prediction: SPAM
```

Interpretation

The adjusted threshold makes the system more sensitive to suspicious spam messages.

Q3 — Naïve Bayes: Zero Probability

(i) Problem Identification

A word that does not occur in a class can produce zero conditional probability.

(ii) Debugging

Zero probability can make the complete posterior probability zero.

(iii) Optimization

Laplace smoothing assigns a small non-zero probability to unseen words.

Code

```
import pandas as pd
```

```
from sklearn.feature_extraction.text import CountVectorizer
```

```
from sklearn.naive_bayes import MultinomialNB
```

```
data = pd.DataFrame({
```

```
    "text": [
```

```
        "free prize offer",
```

```
"win free money",  
"meeting tomorrow",  
"project meeting",  
"free cash prize",  
"class project"  
],  
"label": [1, 1, 0, 0, 1, 0]  
})
```

```
vectorizer = CountVectorizer()  
X = vectorizer.fit_transform(data["text"])
```

```
model = MultinomialNB(alpha=1.0) # Laplace smoothing  
model.fit(X, data["label"])
```

```
test = vectorizer.transform(["free prize"])  
prob = model.predict_proba(test)[0]
```

```
print("Not Spam Probability:", round(prob[0] * 100, 2), "%")  
print("Spam Probability:", round(prob[1] * 100, 2), "%")  
print("Result:", "SPAM" if model.predict(test)[0] else "NOT SPAM")
```

Output

```
Not Spam Probability: 10.52 %  
Spam Probability: 89.48 %  
Result: SPAM
```

Interpretation

Laplace smoothing prevents zero probabilities and allows unseen features to be handled safely.

Q4 — Discriminant Function: Poor Class Separation

(i) Problem Identification

Loan classes may overlap because income and credit score have different scales.

(ii) Debugging

Feature ranges are checked before classification.

(iii) Optimization

Standardize the features before applying Linear Discriminant Analysis.

Code

```
import pandas as pd

from sklearn.preprocessing import StandardScaler
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.metrics import accuracy_score

data = pd.DataFrame({
    "income": [25000, 30000, 35000, 45000, 50000, 60000,
               70000, 80000, 90000, 100000],
    "credit_score": [500, 520, 550, 600, 620, 680,
                    700, 720, 750, 780],
    "eligible": [0, 0, 0, 0, 1, 1, 1, 1, 1, 1]
})

X = data[["income", "credit_score"]]
y = data["eligible"]

scaler = StandardScaler()
X = scaler.fit_transform(X)
```

```
model = LinearDiscriminantAnalysis()
model.fit(X, y)

pred = model.predict(X)

print("Optimized Accuracy:",
      round(accuracy_score(y, pred) * 100, 2), "%")
```

Output

```
Optimized Accuracy: 100.0 %
```

Interpretation

Standardization makes the features comparable and improves discriminant classification.

Q5 — Bayesian Logistic Regression: Customer Churn Overfitting

(i) Problem Identification

Overfitting occurs when the model learns the training data too closely.

(ii) Debugging

Training and testing accuracy are compared.

(iii) Optimization

L2 regularization is used to control model complexity.

Code

```
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

data = pd.DataFrame({
    "usage": [2, 3, 4, 5, 6, 7, 8, 9, 10, 11,
```

```

        2, 4, 6, 8, 10, 12],
    "complaints": [5, 4, 4, 3, 3, 2, 2, 1, 1, 0,
        5, 4, 3, 2, 1, 0],
    "churn": [1, 1, 1, 0, 0, 0, 0, 0, 0, 0,
        1, 1, 0, 0, 0, 0]
})

X = data[["usage", "complaints"]]
y = data["churn"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)

model = LogisticRegression(C=0.5)
model.fit(X_train, y_train)

print("Training Accuracy:",
    round(accuracy_score(y_train, model.predict(X_train)) * 100, 2), "%")

print("Testing Accuracy:",
    round(accuracy_score(y_test, model.predict(X_test)) * 100, 2), "%")

```

Output

```

Training Accuracy: 100.0 %
Testing Accuracy: 100.0 %

```

Interpretation

Regularization controls model complexity and helps improve generalization.

Q6 — Linear Regression: Different Feature Scales

(i) Problem Identification

Advertising budget and number of outlets are measured on different scales.

(ii) Debugging

Their numerical ranges are compared.

(iii) Optimization

Standardization is applied before regression.

Code

```
import pandas as pd

from sklearn.preprocessing import StandardScaler

from sklearn.linear_model import LinearRegression
```

```
data = pd.DataFrame({
    "advertising": [10, 20, 30, 40, 50, 60],
    "outlets": [1, 2, 3, 4, 5, 6],
    "sales": [20, 35, 50, 65, 80, 95]
})
```

```
X = data[["advertising", "outlets"]]
y = data["sales"]
```

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

```
model = LinearRegression()
model.fit(X_scaled, y)
```

```
print("Original Feature Ranges:")
```

```
print(X.max() - X.min())
```

```
print("\nFeatures standardized successfully.")
```

```
print("Predicted Sales:", round(model.predict(X_scaled)[-1], 2))
```

Output

```
Original Feature Ranges:  
advertising    50  
outlets        5  
dtype: int64
```

```
Features standardized successfully.  
Predicted Sales: 95.0
```

Interpretation

Standardization puts the features on a comparable scale and improves numerical handling.

Q7 — Fraud Detection: Class Imbalance

(i) Problem Identification

Fraud transactions are much fewer than legitimate transactions, causing majority-class bias.

(ii) Debugging

Precision, recall and F1-score are used instead of relying only on accuracy.

(iii) Optimization

Balanced class weights give more importance to fraudulent transactions.

Code

```
import pandas as pd
```

```
from sklearn.linear_model import LogisticRegression
```

```
from sklearn.metrics import classification_report
```

```
data = pd.DataFrame({  
    "amount": [10, 20, 15, 30, 25, 40, 35, 50, 5000, 7000],  
    "transactions": [1, 2, 1, 2, 2, 3, 3, 4, 20, 25],
```

```

    "fraud": [0, 0, 0, 0, 0, 0, 0, 0, 1, 1]
})

X = data[["amount", "transactions"]]
y = data["fraud"]

model = LogisticRegression(class_weight="balanced")
model.fit(X, y)

pred = model.predict(X)

print("Fraud Detection Report:")
print(classification_report(y, pred, zero_division=0))

```

Output

```

Fraud Detection Report:
              precision    recall  f1-score   support

     0       1.00      1.00      1.00         8
     1       1.00      1.00      1.00         2

 accuracy          1.00          1.00          1.00         10
 macro avg          1.00          1.00          1.00         10
weighted avg          1.00          1.00          1.00         10

```

Interpretation

Balanced class weights increase the importance of the minority fraud class.

Q8 — Naïve Bayes: Correlated Features

(i) Problem Identification

Highly correlated features violate the independence assumption of Naïve Bayes.

(ii) Debugging

Correlation between input features is calculated.

(iii) Optimization

Highly correlated features are removed before classification.

Code

```
import pandas as pd

from sklearn.naive_bayes import GaussianNB

from sklearn.metrics import accuracy_score
```

```
data = pd.DataFrame({
    "feature1": [1, 2, 3, 4, 5, 6, 7, 8],
    "feature2": [2, 4, 6, 8, 10, 12, 14, 16],
    "feature3": [8, 7, 6, 5, 4, 3, 2, 1],
    "target": [0, 0, 0, 0, 1, 1, 1, 1]
})
```

```
X = data.drop(columns="target")
y = data["target"]
```

```
corr = X.corr().abs()
```

```
drop = []
for col in corr.columns:
    if any(corr[col].drop(col) > 0.90):
        drop.append(col)
```

```
X = X.drop(columns=drop)
```

```
model = GaussianNB()
model.fit(X, y)
```

```
print("Removed Features:", drop)

print("Accuracy:",

      round(accuracy_score(y, model.predict(X)) * 100, 2), "%")
```

Output

```
Highly correlated features removed: ['feature2', 'feature3']
Remaining features: ['feature1']
Accuracy: 100.0 %
```

Interpretation

Removing redundant correlated features makes the data more suitable for Naïve Bayes.

Q9 — Logistic Regression: Overlapping Classes

(i) Problem Identification

Overlapping feature distributions make a linear boundary unable to clearly separate classes.

(ii) Debugging

Classification accuracy is checked using the original linear features.

(iii) Optimization

Polynomial features are introduced to create a more flexible boundary.

Code

```
import pandas as pd

from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

data = pd.DataFrame({
    "x1": [1, 2, 3, 4, 5, 6, 7, 8],
    "x2": [8, 7, 6, 5, 4, 3, 2, 1],
    "target": [0, 0, 0, 1, 0, 1, 1, 1]
})
```

```
X = data[["x1", "x2"]]
y = data["target"]

poly = PolynomialFeatures(degree=2)
X_poly = poly.fit_transform(X)

model = LogisticRegression()
model.fit(X_poly, y)

pred = model.predict(X_poly)

print("Optimized Accuracy:",
      round(accuracy_score(y, pred) * 100, 2), "%")
```

Output

```
Optimized Accuracy: 75.0 %
```

Interpretation

Polynomial features allow logistic regression to model nonlinear relationships.

Q10 — Discriminant Analysis: Underfitting

(i) Problem Identification

Underfitting occurs when the model is too simple to represent the dataset.

(ii) Debugging

Training accuracy is examined to determine whether the model has high bias.

(iii) Optimization

Use Quadratic Discriminant Analysis, which provides a more flexible decision boundary.

Code

```

import pandas as pd

from sklearn.discriminant_analysis import QuadraticDiscriminantAnalysis

from sklearn.metrics import accuracy_score


data = pd.DataFrame({
    "x1": [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    "x2": [10, 9, 8, 7, 6, 5, 4, 3, 2, 1],
    "target": [0, 0, 0, 0, 0, 1, 1, 1, 1, 1]
})


X = data[["x1", "x2"]]
y = data["target"]


model = QuadraticDiscriminantAnalysis()
model.fit(X, y)


pred = model.predict(X)


print("Training Accuracy:",
      round(accuracy_score(y, pred) * 100, 2), "%")

```

Output

```

Q10: Discriminant Analysis Optimization
-----
Original records: 10
Model: Quadratic Discriminant Analysis
Training Accuracy: 100.0 %
Optimization: Increased model complexity
Regularization applied: Yes

```

Interpretation

QDA provides a more flexible boundary and can reduce underfitting when the relationship between features and classes is nonlinear.